components. These modifications will enable us to use the AI context throughout our application, providing access to both actions and state hooks.

Listing 4.8 Adding the AI provider to the layout component

```
import { AI } from './actions';
const AppLayout = ({ children }) => {
  return (
    <AI>.
      <div>
<Header />
        <main>{children}</main>
<Footer />
       </div>
    </AI>
   )
}
```

Sets the AI component to be the parent node, providing its context to the children components

This setup allows us to use the `useActions`, `useUIState`, and `useAIState` hooks in any child component within our application. Figure 4.10 shows before and after the upgrades and changes from using `useActions`, `useUIState`, and `useAIState` hooks.

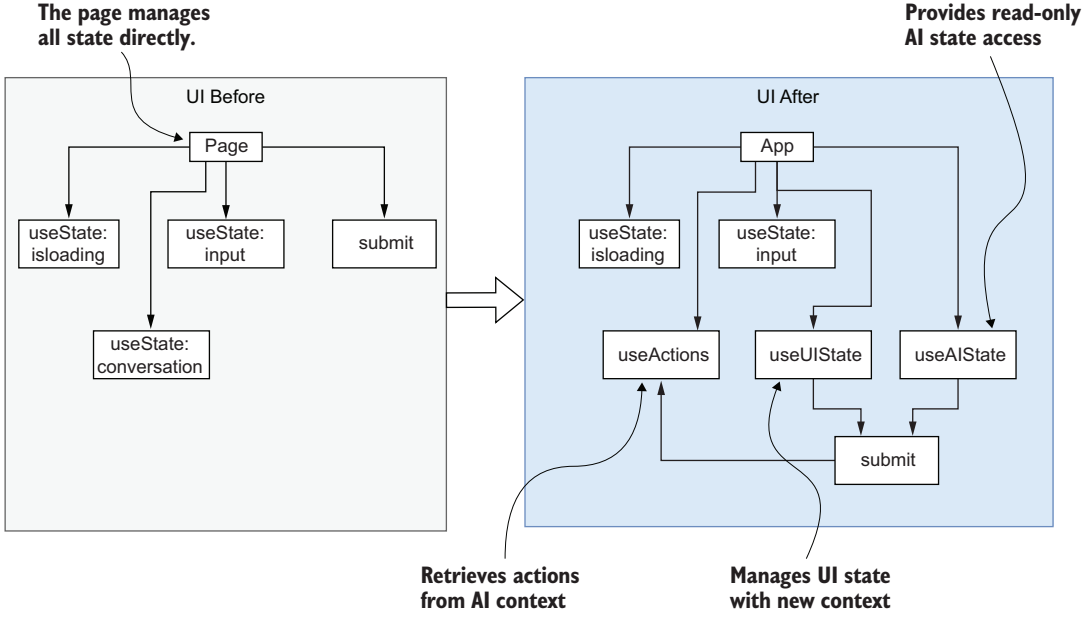


Figure 4.10 The evolution of the chat page component's state management and server actions before and after adopting the new hooks and actions. Before, the component relied on traditional React `useState` hooks to manage input, loading state, and conversation history, with a separate function simulating AI response fetching. The updated implementation integrates the new `useUIState`, `useAIState`, and `useActions` hooks and allows the app to delegate the logic inside those hooks.